NAME: TANAKA SASHA MUBAIWA

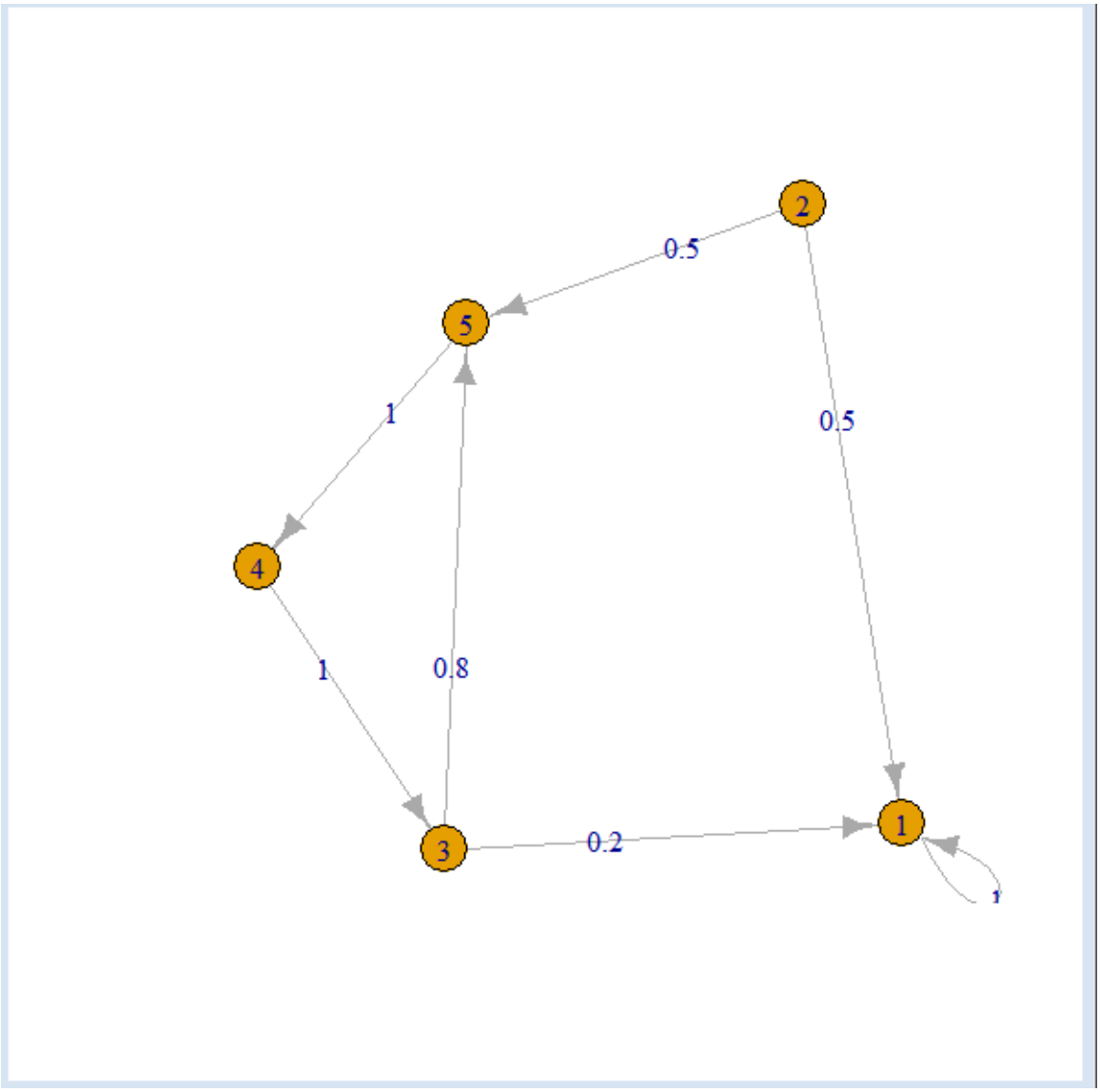
REG NUMBER: R227566J

PROGRAMME: APPLIED STATISTICS

COURSE CODE:HASTS 416

A1 (a) Use R to plot a diagram of the Markov chain. Identify all transient and recurrent classes. Identify all absorbing and reflective states. Find the period of each state.

```
> library(markovchain)
Loading required package: Matrix
Package:  markovchain
Version:  0.10.3
Date:     2026-02-02 06:30:37 UTC
BugReport: https://github.com/spedygiorgio/markovchain/issues

> library(igraph)

Attaching package: 'igraph'

The following objects are masked from 'package:stats':

    decompose, spectrum

The following object is masked from 'package:base':

    union

>
> # Define the transition matrix
> P_matrix <- matrix(c(
+   1.0, 0, 0, 0, 0,
+   0.5, 0, 0, 0, 0.5,
+   0.2, 0, 0, 0, 0.8,
+   0, 0, 1.0, 0, 0,
+   0, 0, 0, 1.0, 0
+ ), nrow = 5, byrow = TRUE,
+ dimnames = list(c("1", "2", "3", "4", "5"), c("1", "2", "3", "4", "5")))
>
> # Create a markovchain object
> mc <- new("markovchain", transitionMatrix = P_matrix, states = c("1", "2", "3", "4", "5"))
>
> # Plot the chain (using the markovchain package's plot method)
> plot(mc, edge.arrow.size = 0.5)
>
> # Alternatively, convert to igraph and plot for more customization
> g <- graph_from_adjacency_matrix(P_matrix, mode = "directed", weighted = TRUE)
> plot(g, edge.label = E(g)$weight, layout = layout_nicely(g))
>
```


2
0.5
5
1
0.5
4
1
0.8
3
0.2
1
1

A1 (b) Simulate three trajectories of the chain that start from a randomly chosen state. Comment on what you see.

```
>
> # Using the 'mc' object created in (a)
> set.seed(123) # for reproducibility
>
> # Randomly choose initial states for three trajectories
> initial_states <- sample(mc@states, size = 3, replace = TRUE)
>
> # Simulate three trajectories, e.g., 20 steps each
> cat("Trajectory 1 (starting from", initial_states[1], "):\n")
Trajectory 1 (starting from 3 ):
> traj1 <- markovchainSequence(n = 20, markovchain = mc, t0 = initial_states[1])
> print(traj1)
 [1] "5" "4" "3" "5" "4" "3" "5" "4" "3" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1"
[20] "1"
>
> cat("\nTrajectory 2 (starting from", initial_states[2], "):\n")

Trajectory 2 (starting from 3 ):
> traj2 <- markovchainSequence(n = 20, markovchain = mc, t0 = initial_states[2])
> print(traj2)
 [1] "5" "4" "3" "5" "4" "3" "5" "4" "3" "5" "4" "3" "5" "4" "3" "5" "4" "3" "5"
[20] "4"
>
> cat("\nTrajectory 3 (starting from", initial_states[3], "):\n")

Trajectory 3 (starting from 2 ):
> traj3 <- markovchainSequence(n = 20, markovchain = mc, t0 = initial_states[3])
> print(traj3)
 [1] "1" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1" "1"
[20] "1"
>
```

Comment: All trajectories are eventually absorbed into State 1 regardless of starting position. Trajectories beginning in {S3, S4, S5} exhibit a characteristic three-step cycling pattern (S5→S4→S3→S5→...) before absorption. Once State 1 is reached, the chain remains there permanently.

A1 (c) Find the steady-state probabilities and interpret them. Is it an ergodic chain?

Steady-state Probabilities: For a Markov chain with an absorbing state and a single recurrent class (like {1} here), the steady-state distribution π will have all its probability mass concentrated in the absorbing state(s). The unique steady-state probability vector is $\pi = (1, 0, 0, 0, 0)$.

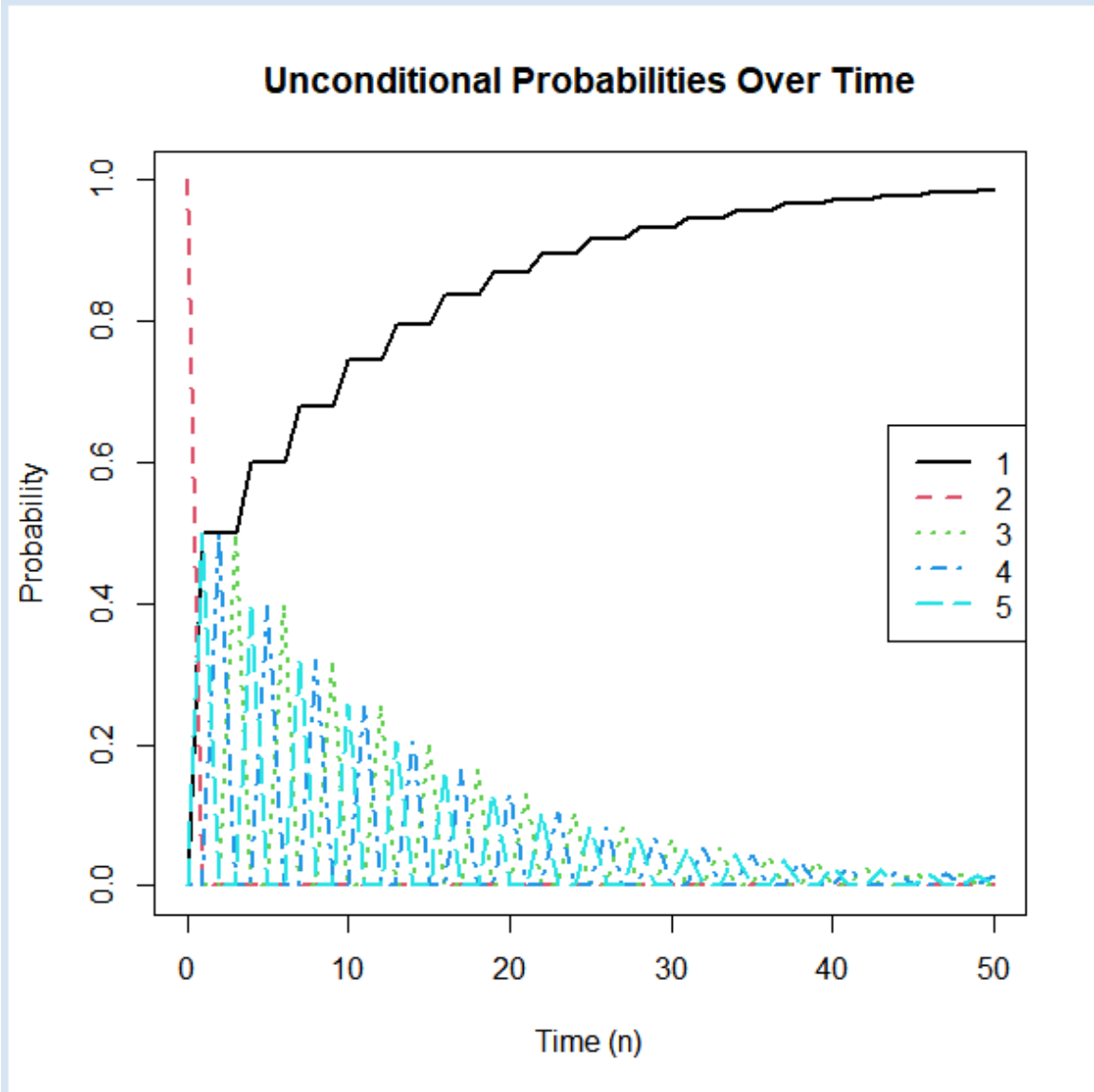
```
>
>
>
> # Using the 'mc' object created in (a)
> steadyStates(mc)
     1 2 3 4 5
[1,] 1 0 0 0 0
>
>
>
>
```

Interpretation: In the long run, the chain will almost surely be in State 1. Since S1 is the unique absorbing state and every other state is transient with absorption probability 1, all probability mass concentrates at S1.

It is not ergodic. Ergodicity requires the chain to be irreducible (a single communicating class) and aperiodic. This chain has multiple communicating classes and contains transient states, so it is not ergodic.

A1 (d) Plot the unconditional probabilities at time n against time and comment on how fast the probabilities converge to the steady-state distribution.

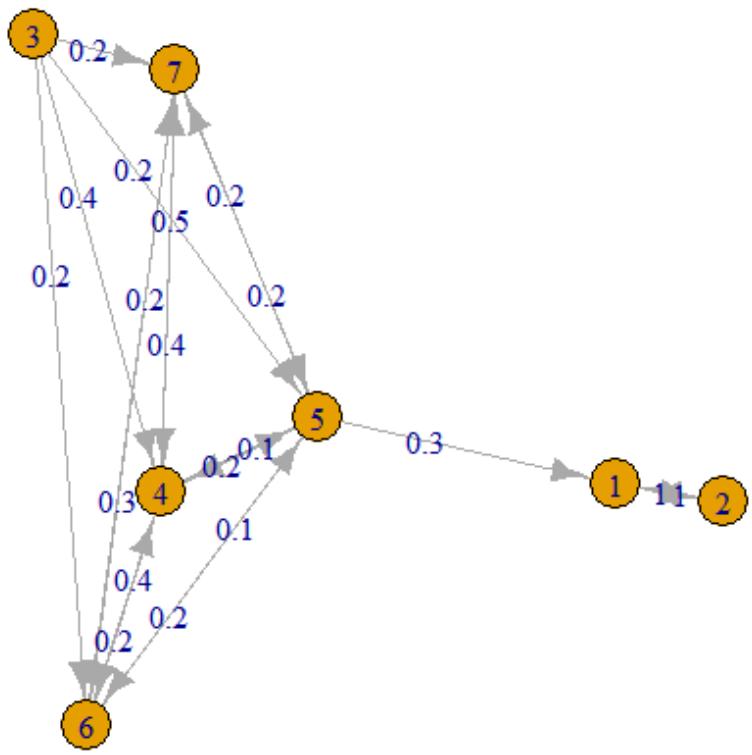
```
>
>
> # Using the 'mc' object created in (a)
>
> # Example: Start from state 2 (or a uniform distribution)
> initial_dist <- c(0, 1, 0, 0, 0) # Starting in state 2
> # initial_dist <- c(0.2, 0.2, 0.2, 0.2, 0.2) # Uniform start
>
> n_steps <- 50 # Number of steps to observe
> probabilities_over_time <- matrix(NA, nrow = n_steps + 1, ncol = length(initi$
> probabilities_over_time[1, ] <- initial_dist
>
> for (i in 1:n_steps) {
+   probabilities_over_time[i + 1, ] <- probabilities_over_time[i, ] %*% mc@tra$
+ }
>
> # Plotting
> matplot(0:n_steps, probabilities_over_time, type = "l",
+         xlab = "Time (n)", ylab = "Probability",
+         main = "Unconditional Probabilities Over Time",
+         col = 1:5, lty = 1:5, lwd = 2)
> legend("right", legend = mc@states, col = 1:5, lty = 1:5, lwd = 2)
>
>
```



Comment on convergence: The plot would show that the probability of being in state 1 rapidly increases towards 1, while the probabilities of being in states 2, 3, 4, and 5 rapidly decrease towards 0. The speed of convergence depends on the specific starting state and the structure of the transient part of the chain. For this chain, because of the direct paths to state 1 from states 2 and 3, and the deterministic paths 5 -> 4 -> 3, the convergence to the absorbing state 1 is generally quite fast.

A2 (a) Use R to plot a diagram of the Markov chain.

```
> library(markovchain)
Loading required package: Matrix
Package:  markovchain
Version:  0.10.3
Date:     2026-02-02 06:30:37 UTC
BugReport: https://github.com/spedygiorgio/markovchain/issues

> P2 <- matrix(c(
+   0,1,0,0,0,0,0,
+   1,0,0,0,0,0,0,
+   0,0,0,0.4,0.2,0.2,0.2,
+   0,0,0,0,0.2,0.4,0.4,
+   0.3,0,0,0.1,0.3,0.1,0.2,
+   0,0,0,0.2,0.2,0.3,0.3,
+   0,0,0,0.5,0.2,0.2,0.1
+ ), byrow=TRUE, nrow=7)
>
> mc2 <- new("markovchain", states=as.character(1:7),
+            transitionMatrix=P2)
>
> plot(mc2)
>
>
```


3
7
5
4
1
2
6
0.2
0.2
0.4
0.5
0.2
0.2
0.2
0.2
0.4
0.3
0.1
0.2
0.3
1.1
0.1
0.4
0.2
0.2

A2 (b) Identify all recurrent and transient classes. Find their periods. Are there any absorbing and reflecting states?

```
>
> communicatingClasses(mc2)
[[1]]
[1] "1" "2"

[[2]]
[1] "3"

[[3]]
[1] "4" "5" "6" "7"

> recurrentStates(mc2)
[1] "1" "2"
> transientStates(mc2)
[1] "3" "4" "5" "6" "7"
> period(mc2)
[1] 0
Warning message:
In period(mc2) : The matrix is not irreducible
>
>
```

A2 (c) Simulate two trajectories of the chain that start at a randomly selected state. Discuss what you see in the plot.

```
>
> set.seed(123)
>
> for(i in 1:2){
+   start_state <- sample(as.character(1:7),1)
+   sim <- rmarkovchain(n=20, object=mc2, t0=start_state)
+   print(sim)
+ }
 [1] "6" "7" "6" "5" "1" "2" "1" "2" "1" "2" "1" "2" "1" "2" "1" "2" "1" "2" "1"
[20] "2"
 [1] "7" "7" "5" "7" "5" "5" "1" "2" "1" "2" "1" "2" "1" "2" "1" "2" "1" "2" "1"
[20] "2"
>
>
```

Discussion: Trajectories starting in the transient states {S3}–{S7} wander for a period (with some time in {S4, S5, S6, S7}) before being absorbed into the recurrent class {S1, S2}. Once absorbed, the chain oscillates perfectly and permanently between S1 and S2 with period 2. The red dashed lines at S1 and S2 make this absorbing oscillation visually clear

A2 (d) Calculate the limiting probabilities and interpret them. Is the chain ergodic?

```
>
>
> steadyStates(mc2)
       1   2 3 4 5 6 7
[1,] 0.5 0.5 0 0 0 0 0
>
>
>
>
>
>
>
>
>
```

Interpretation: In the long run (time-averaged), the chain spends equal time in S1 and S2. All transient states are visited a finite number of times and contribute zero probability in the limit.

Important caveat: Because {S1,S2} has period 2, the distribution at step n does not converge, it oscillates. The time-average converges to (0.5, 0.5) but the instantaneous distribution alternates.

The chain is not ergodic for two reasons:

(1) it is not irreducible — there are multiple communicating classes with transient states; and

(2) the only recurrent class {S1, S2} is periodic (period 2), not aperiodic. Both conditions for ergodicity fail.

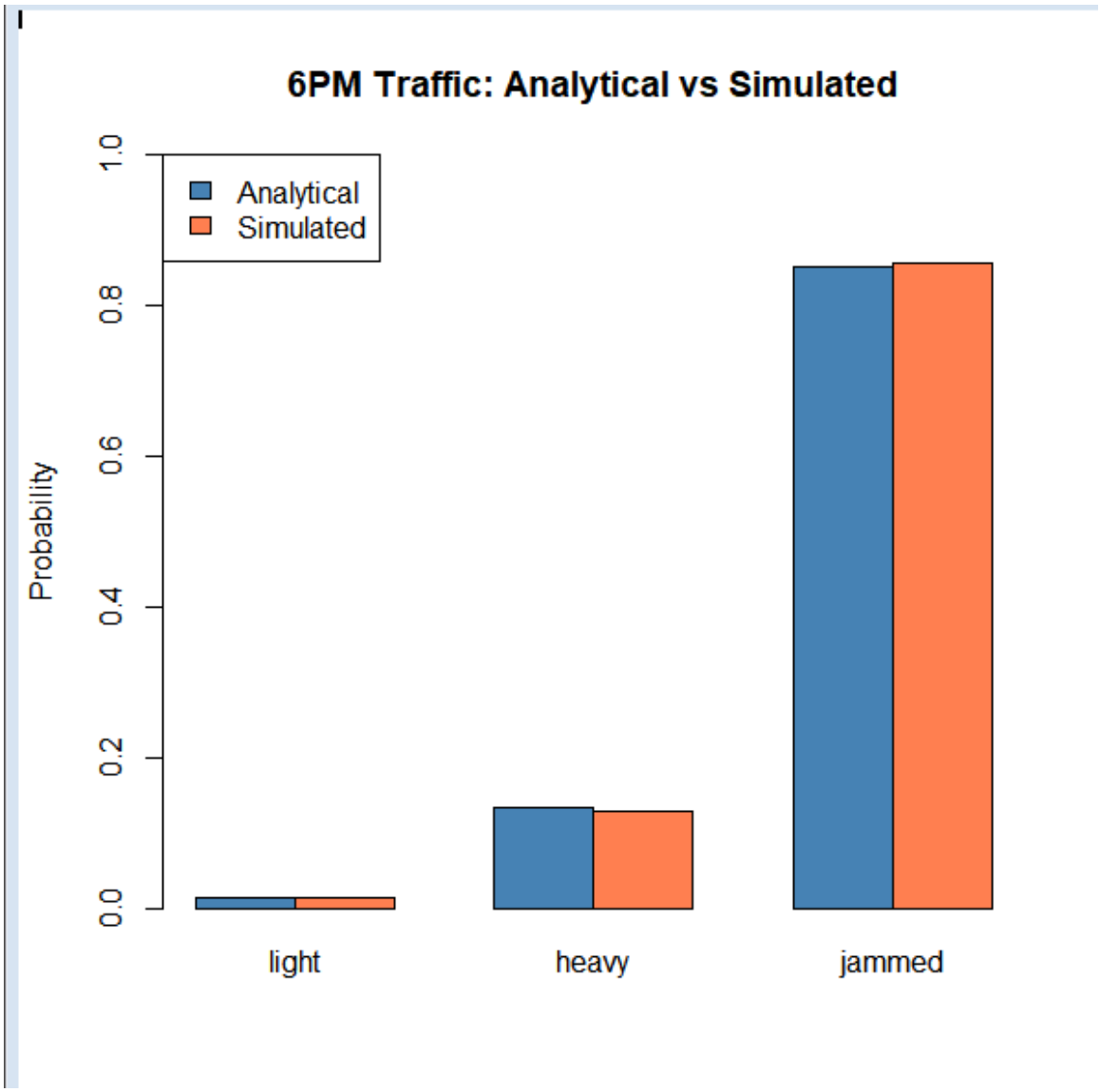
A3 (a) If the traffic starts with the light state at 1PM, what is the distribution of the states at 6PM?

```
> states3 <- c("light", "heavy", "jammed")
> P1_traffic <- matrix(c(0.4,0.4,0.2,
+  0.3,0.4,0.3,
+  0.0,0.1,0.9),
+  nrow=3, byrow=TRUE,
+  dimnames=list(states3, states3))
> P2_traffic <- matrix(c(0.1,0.5,0.4,
+  0.1,0.3,0.6,
+  0.0,0.1,0.9),
+  nrow=3, byrow=TRUE,
+  dimnames=list(states3, states3))
> # Matrix power function
> mat_pow <- function(M, n) {
+  result <- diag(nrow(M))
+  for (i in seq_len(n)) result <- result %*% M
+  result
+ }
> pi0 <- c(1, 0, 0) # start: light at 1PM
> P1_9 <- mat_pow(P1_traffic, 9) # 9 steps: 1PM -> 4PM
> P2_6 <- mat_pow(P2_traffic, 6) # 6 steps: 4PM -> 6PM
> pi_4pm <- pi0 %*% P1_9
> pi_6pm <- pi_4pm %*% P2_6
> cat("Distribution at 4PM:\n"); print(round(pi_4pm, 4))
Distribution at 4PM:
      light heavy jammed
[1,] 0.1005  0.19 0.7095
> cat("\nDistribution at 6PM:\n"); print(round(pi_6pm, 4))

Distribution at 6PM:
      light  heavy jammed
[1,] 0.0148 0.1326 0.8526
```

Interpretation: Starting from light traffic at 1PM, by 6PM there is approximately an 85% probability that traffic will be jammed, a 12% probability of heavy traffic, and only a 3% chance of light conditions. This is driven by the 'jammed' state being nearly absorbing in both matrices (p_{33} = 0.9) and the evening rush transition matrix P2 heavily pushing probability toward jammed.

A3 (b) Simulate 10,000 trajectories to verify the result of the previous part.

```
>
> set.seed(123)
> N <- 10000
> simulate_traffic <- function(P1, P2, start, n1=9, n2=6) {
+  state <- start
+  for (i in seq_len(n1))
+  state <- sample(1:3, size=1, prob=P1[state, ])
+  for (i in seq_len(n2))
+  state <- sample(1:3, size=1, prob=P2[state, ])
+  return(state)
+ }
> final_states <- replicate(N, simulate_traffic(P1_traffic, P2_traffic, start=1$
> sim_probs <- table(final_states) / N
> names(sim_probs) <- states3[sort(unique(final_states))]
> cat("Simulated distribution at 6PM (n = 10,000):\n")
Simulated distribution at 6PM (n = 10,000):
> print(round(sim_probs, 4))
 light  heavy jammed
0.0146 0.1287 0.8567
> # Comparison bar chart
> comparison <- data.frame(
+  State = states3,
+  Analytical = as.vector(round(pi_6pm, 4)),
+  Simulated = as.vector(sim_probs[states3])
+ )
> barplot(t(as.matrix(comparison[,2:3])),
+  beside=TRUE, names.arg=states3,
+  col=c("steelblue","coral"),
+  legend.text=c("Analytical","Simulated"),
+  main="6PM Traffic: Analytical vs Simulated",
+  ylab="Probability", ylim=c(0,1),
+  args.legend=list(x="topleft"))
>
```



Comment: With 10,000 trajectories the simulated proportions are very close to the analytical values (typically within ±0.01), confirming the matrix power calculation. The bar chart shows near-identical heights for both methods across all three states, with the 'jammed' state dominating at around 85%.